Laboratorio 2 — Semáforos en XV6

Clase 7: entrega final y cierre

Sistemas Operativos — FaMAF, UNC

06/10/2026

Agenda

- La entrega final
- Si algo falla
- El punto estrella
- El video
- Repaso del lab

La entrega final

Para el 07/10 a las 23:59

- ☐ Lo de la intermedia andando: las 4 syscalls y `pingpong`.
- ☐ `user/prodcons.c` en `UPROGS`, imprimiendo `produce K pid=P` y `consume pid=P`.
- ☐ `make grade-final` → **15/15** (y `grade-parcial` sigue en 11/11).
- ☐ Video de hasta **7 minutos**.
- ☐ `make clean`, commit y push.

Se corrige la rama por defecto del repo (`riscv`)

Si trabajaron en otra rama, mergeen antes del 07/10. Lo que no esté en `riscv` no se ve.

`make grade-final`, test por test

Programa	Tests	Pts
`semtest`	noinit · init value · closeuse · exhaust reuse · grow beyond initial · double open rejected	6
`pingpong`	no parameters · wrong parameters · three rounds · order and termination · concurrency	5
`prodcons`	no parameters · wrong parameters · B=1 alternating · B=3 capacity	4
estrella	`prodcons 2 20 2 2`	0

- Los 11 primeros son los de la intermedia: si alguno se rompió al agregar `prodcons`, es una regresión.

Qué miran los tests de prodcons

- Sin argumentos, o con argumentos inválidos (`prodcons 0 5`): tiene que aparecer una línea que **empiece** con `ERROR`.
- Cuentan las **líneas** con `produce` y con `consume`: tiene que haber N de cada una.
- Recorren la salida llevando la cuenta de ítems en vuelo: nunca más de B, y 0 al terminar.
- Corren `prodcons B N; echo DONE`: después del `DONE` no puede aparecer nada.

Una línea partida no cuenta como `produce` ni como `consume`. Y si el padre termina antes que el hijo, el shell sigue con el echo y el hijo imprime **después** del `DONE`.

Antes del autograder, a mano

```
$ prodcons 1 1                    el mínimo
$ prodcons 1 20                   buffer chico, muchos ítems
$ prodcons 8 8                    B == N
$ prodcons 4 1000                 estrés
$ pingpong 50                     regresión
$ pingpong 5 & ; prodcons 3 10    los dos a la vez
```

- Si algo falla, la salida a mano les dice **qué** pasó; el autograder sólo les dice **que** pasó.

Si algo falla

Síntoma → dónde mirar

Síntoma	**Dónde mirar**
se cuelga	un `down` sin su `up`, o `empty`/`full` mal inicializados
más de B producidos sin consumir	¿`empty` arranca en B?
faltan o sobran ítems	los loops de N, en el productor y en el consumidor
líneas partidas	el `printf` quedó fuera del mutex
algo después del `DONE`	el padre no espera al hijo
anda solo, falla con otro al lado	ids de semáforo fijos
falta el `pid=`	no rompe los tests, pero el enunciado lo pide

No fijen los ids de los semáforos

```
#define MUTEX 5
```

- Anda mientras `prodcons` corre solo.
- Con `pingpong 5 & ; prodcons 3 10` —lo que pide el video— los dos programas pueden querer el **mismo** id: uno falla en `sem_open`, o peor, se traban entre ellos.
- Ya estaba en la tabla de la clase 4: anda solo, falla en concurrencia.

La política del lab ya les da la herramienta: `sem_open` **falla si el id está abierto**.

El punto estrella

prodcons B N P C, en pseudocódigo

```
cada uno de los P productores:
  por cada ítem que le toca:
    down(empty)
    down(mutex)
      imprimir produce
    up(mutex)
    up(full)
```

```
cada uno de los C consumidores:
  por cada ítem que le toca:
    down(full)
    down(mutex)
      imprimir consume
    up(mutex)
    up(empty)
```

- **Repartir:** entre todos, exactamente N. Cada consumidor tiene que saber cuántos le tocan: si no, el último se queda esperando un ítem que nunca llega.
- **Terminar:** el padre espera a los P+C hijos, y recién ahí cierra los semáforos.

El video

El video: hasta 7 minutos, en cinco partes

Parte	**Qué tiene que estar**
I. Introducción	el equipo, las 4 syscalls, las decisiones de diseño y **por qué**
II. Invocación	de modo usuario a modo kernel; los argumentos en `a0`, `a1`, …; las políticas de error
III. Concurrencia	`acquire`/`release`; `sem_down` con `sleep`, `sem_up` con `wakeup`; dos programas a la vez
IV. Programas	`pingpong`; `prodcons` con $B > 1$ y los pids; la estrella si la hicieron; la corrida concurrente
V. Cierre	

La corrida concurrente

```
$ pingpong 5 & ; prodcons 3 10
ping
pong
ping
pong
propduicneg
1 ppido=2n6
gp
roducep i2n gp
id=2p6
oconnsgum
...
produce 4 pid=26
consume pid=30
```

- Usan semáforos **distintos**: nada ordena a un programa respecto del otro.
- `printf` hace un `write` **por carácter**, y xv6 corre en 3 CPUs: se mezclan letra por letra.
- Está bien. Lo que tiene que valer es el orden **dentro** de cada programa.

Muéstrenla y explíquenla: es justo lo que pide la parte III, "dos programas a la vez".

Qué suma y qué resta

Suma

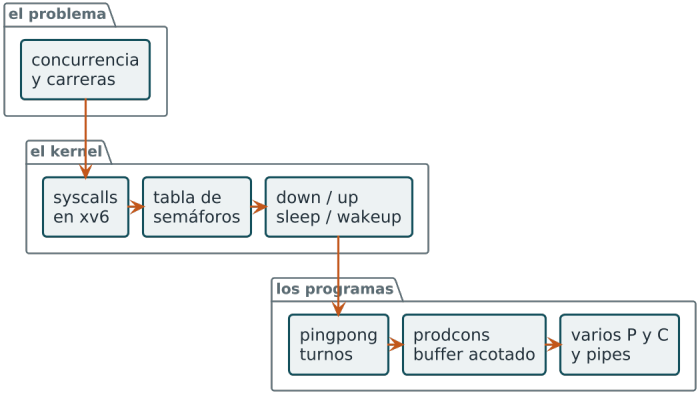
- Explicar decisiones: por qué `while` y no `if`, qué pasa si cierran un semáforo con procesos esperando.
- Mostrar el código clave, sin scrollear de más.
- Terminal con fuente grande, audio claro.
- Cada integrante, una parte.

Resta

- Mostrar sólo `make grade` pasando.
- Leer código sin explicarlo.
- No mencionar `sleep`/`wakeup`.
- Confundir `sem_open` con el `open` de archivos.
- Pasarse de los 7 minutos.

Repaso del lab

El recorrido



- Arrancaron con un problema de usuario y lo resolvieron adentro del kernel: los semáforos que usan `pingpong` y `prodcons` los escribieron ustedes.

Cierre

Lo mismo, en todos lados

- **POSIX:** `sem_open`, `sem_wait`, `sem_post`. Semáforos **con nombre**, como los suyos.
- **Java:** `java.util.concurrent.Semaphore`.
- **Go:** un canal con buffer, `make(chan T, B)`, es un buffer acotado de B lugares.
- Y el pipe del shell, que vimos la clase pasada.

Lo que aprendieron no es xv6: es cómo coordinar procesos que comparten algo.

Para seguir leyendo

- **OSTEP**, cap. 31, *Semaphores*.
  `https://pages.cs.wisc.edu/~remzi/OSTEP/threads-sema.pdf`
- **xv6 book** (rev3): cap. 4 *Traps and system calls*, cap. 6 *Locking*, y §7.5–7.7.
  `https://pdos.csail.mit.edu/6.1810/2022/xv6/book-riscv-rev3.pdf`
- `man 7 sem_overview`, en cualquier Linux.

Entrega final: 07/10 23:59

¿Preguntas?